2025 산학프로젝트
최종 결과보고

레이저 변위센서를 이용한 곡률측정 시스템

쏴조빔 팀

지도교수:송동섭 교수님

참여기업:㈜천진

2025.12.19.(금)

목차

1. 프로젝트 제작동기
2. 프로젝트 제작과정
3. 작품설명
4. 프로젝트 결과
5. 결과분석 및 토의
6. 느낀점 및 애로사항

01 프로젝트 제작동기

공정 불량의 원인

디스플레이 및 반도체 제조 공정에서 평탄도 및 미세한 곡률 변화는 심각한 품질 문제를 초래

- ✓ **측정 반복성 저하**: 동일 공정 내 데이터 신뢰도 하락
- ✓ **공정 편차 증가**: 제품 간 품질 격차 발생
- ✓ **제품 균일도 저하**: 공정 불안정 유발

→ 을 **형상 변화를 실시간으로 시각화 할 수 있는 시스템 목표로 개발을 진행**

성 명	소속학과	역할 및 수행내용
김재민	전자공학과	하드웨어
이규현	전자공학과	하드웨어
이준서	전자공학과	하드웨어
우희준	전자공학과	소프트웨어
주재홍	전자공학과	소프트웨어
조민재	전자공학과	소프트웨어

01 프로젝트 제작동기

⚠ 기존 방식의 한계

1. 접촉식 방식의 제품 손상 위험

압력에 의해 제품 표면에 미세한 스크래치 또는 변형 유발 가능

2. 단일 지점 측정의 신뢰성 부족

장비 규모가 크고 구축 비용이 높아 현장 적용에 부담



제안: 원호 스캔 시스템

1. 단일 모터 기반 연속 스캔

모터 회전만으로 넓은 범위 측정 가능

2. 실시간 시각화

형상 변화를 즉각적으로 그래프화 하여 공정 피드백 속도 향상

02 프로젝트 제작과정

STEP 1



시스템 기획 및 설계

- **메커니즘 설계**
  단일 모터 기반 원호 스캔 구조
  (측정 범위: -30˚ ~ 30˚)
- **데이터 설계**
  센서 거리 데이터와 모터 각도 데이터의 실시간 동기화 구조 설계

STEP 2



하드웨어(H/W) 구축

- **핵심 부품 선정**
  고정밀 레이저 변위 센서 및 모터 선정
- **회로 구성**
  STM32 MCU 기반 제어 회로 및 펌웨어 개발

STEP 3



소프트웨어(S/W) 구축

- **알고리즘 구현 (STM32)**
  거리값 변환 및 곡률/평탄도 연산 및 알고리즘 구현
- **GUI 개발(Python)**
  측정 결과(곡률, 평탄도) 출력 및 실시간 시각화 GUI 구현

STEP 4

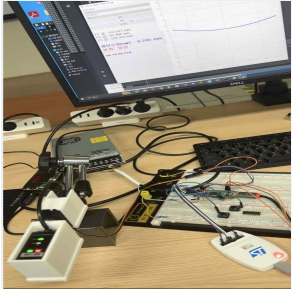


통합 및 성능 검증

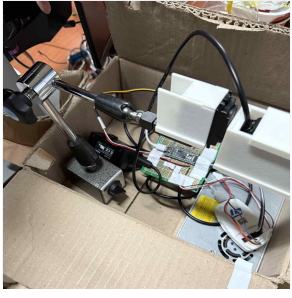
- **시스템 통합**
  H/W 구동과 S/W 데이터 처리의 최적화 및 연동
- **성능 평가**
  반복 테스트를 통한 정밀도 검증 및 기존 방식과의 성능 비교

호서대학교
반도체특성화대학

02 프로젝트 제작과정



제작 초기



제작 중반



최종 완성

03 작품설명



1. 초기화

모터 원점 복귀 및
센서 영점 조정 수행



2. 연속 스캔

-30° ~ 30° 구간을 연속
스캔하며 데이터 획득



3. 데이터 연산

스캔 데이터 기반으로
곡률 및 평탄도 연산



4. 결과 출력

GUI를 통해 결과
및 그래프 출력

03 작품설명

Hardware

Sensor	Res. 4μm
Motor	Range ±30˚
MCU	STM32F411CEU6
Mount	3D Printed

Software

Interface	Python GUI
Comm.	UART Serial
Logic	Bow / Curv

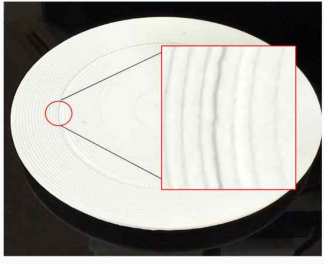
동작 프로세스(Operation Process)

1. **제어 시작(Start)**
   PC GUI 측정 명령 전송 및 모터/센서 초기화
2. **스캔(Scan)**
   -30˚ ~ 30˚ 회전, **30ms(33Hz)** 주기 측정
3. **연산(MCU Calculation)**
   STM32 내부에서 거리/각도 데이터 동기화 및 처리
4. **판정 및 분석(Analysis)**
   Python 내에서 임계값 **0.05mm** 초과 시 **불량(NG)** 자동 판정
5. **결과 시각화(Visualization)**
   Python GUI에서 결과 및 그래프 출력

03 작품설명



측정 재료



재료 표면

03 작품설명

STM32 코드

실시간 거리 측정 및 전송

```
if (current_tick - g_rtd_timer >= 100)
{
    g_rtd_timer = current_tick;
    distance_mm = measure_distance();

    int32_t rtd_int = (int32_t)(distance_mm * 1000.0f);
    char rtd_buffer[30];
    int len = snprintf(rtd_buffer, sizeof(rtd_buffer), "RTD,%ld\r\n", rtd_int);
    HAL_UART_Transmit(&huart1, (uint8_t*)rtd_buffer, len, 100);
}
```

거리 변환

```
float measure_distance(void)
{
    #define OVERSAMPLE_COUNT 8
    uint32_t adc_sum = 0;

    for (int i = 0; i < OVERSAMPLE_COUNT; i++)
    {
        HAL_ADC_Start(&hadc1);
        if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK)
        {
            adc_sum += HAL_ADC_GetValue(&hadc1);
        }
        HAL_ADC_Stop(&hadc1);
    }

    float avg_adc_value = ((float)adc_sum / (float)OVERSAMPLE_COUNT;
    float measured_voltage = ((float)avg_adc_value * (3.3f / 4095.0f);
    float sensor_voltage = measured_voltage * (3.2f + 3.3f) / 3.3f;
    return (10.0f * sensor_voltage) * 123.1f;
}
```

스캔시 모터제어

```
if (g_scan_state == 0)
{
}
else if (g_scan_state == 1)
{
    if (g_scan_timer == 0)
    {
        g_current_pulse = PULSE_MIN;
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, g_current_pulse);
        g_scan_timer = current_tick;
    }
    if (current_tick - g_scan_timer > 2000)
    {
        g_scan_state = 2;
        g_scan_timer = current_tick;
    }
}
else if (g_scan_state == 2)
{
    if (current_tick - g_scan_timer >= 30)
    {
        g_scan_timer = current_tick;
        float scan_dist = measure_distance();
```

UART 통신

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1)
    {
        if (g_uart_rx_byte == 'M' && g_scan_state == 0)
        {
            g_scan_state = 1;
            g_scan_timer = 0;
        }
        HAL_UART_Receive_IT(&huart1, &g_uart_rx_byte, 1);
    }
}
```

스캔 데이터 전송

```
int32_t dist_int = (int32_t)(scan_dist * 1000.0f);
char scan_buffer[50];
int len = snprintf(scan_buffer, sizeof(scan_buffer), "SCAN,%d,%ld\r\n", g_current_pulse, dist_int);
HAL_UART_Transmit(&huart1, (uint8_t*)scan_buffer, len, 100);
```

평탄도 및 곡률 계산

```
const float cos30 = cosf(30.0f * M_PI / 180.0f);
const float sin30 = sinf(30.0f * M_PI / 180.0f);

float y_left = distance_storage[0] * cos30;
float y_right = distance_storage[1] * cos30;
float y_side_avg = (y_left + y_right) / 2.0f;
bow_value_mm = y_side_avg - distance_storage[1] - BOW_OFFSET;

float x1 = -distance_storage[0] * sin30;
float y1 = distance_storage[0] * cos30;
float x2 = 0.0f;
float y2 = distance_storage[1];
float x3 = distance_storage[2] * sin30;
float y3 = distance_storage[2] * cos30;

float a = sqrtf(powf(x1-x2, 2) + powf(y1-y2, 2));
float b = sqrtf(powf(x2-x3, 2) + powf(y2-y3, 2));
float c = sqrtf(powf(x3-x1, 2) + powf(y3-y1, 2));
float area = 0.5f * fabsf(x1*(y2-y3) + x2*(y3-y1) + x3*(y1-y2));

if (a*b*c != 0) curvature_val = (4.0f * area) / (a*b*c);
else curvature_val = 0.0f;
if (bow_value_mm < 0) curvature_val = -curvature_val;
```

03 작품설명

Python 코드

```
graph_panel = QVBoxLayout()
```

그래프 출력

```
self.plot_profile = pg.PlotWidget(title="그래프(graph)")
self.plot_profile.setLabel("bottom", "Width (mm)")
self.plot_profile.setLabel('left', "Height (mm)")
self.plot_profile.showGrid(x=True, y=True)
self.plot_profile.setAspectLocked(True)
self.plot_profile.invertY(True)
self.curve_profile = self.plot_profile.plot(
    pen=None, symbol='o', symbolSize=5, symbolBrush='b'
)
```

```
THRESH = 0.05
mask = residuals > THRESH
```

임계값 초과시 빨간점 표시

```
bad_x = x_data[mask]
bad_y = y_data[mask]
self.curve_profile_bend.setData(bad_x, bad_y)
print(f"[MA Highlight] Found {len(bad_x)} outlier points.")
```

```
ctrl_panel = QVBoxLayout()
self.port_combo = QComboBox()
self.baud_combo = QComboBox()
self.baud_combo.addItems(["9600", "115200"])

self.btn_connect = QPushButton("연결")
self.btn_connect.clicked.connect(self.toggle_connection)

self.btn_scan = QPushButton("스캔 시작")
self.btn_scan.clicked.connect(self.start_scan)
self.btn_scan.setEnabled(False)

self.btn_clear = QPushButton("그래프 초기화")
self.btn_clear.clicked.connect(self.clear_graphs)

self.lbl_status = QLabel("상태: 대기중")
self.lbl_status.setStyleSheet("color: gray; font-weight: bold;")

lbl_style = "font-size: 14pt; font-weight: bold; margin-top: 10px;"
```

GUI 구현

```
self.plot_rtd = pg.PlotWidget(title="실시간 거리 센서 값")
self.plot_rtd.setLabel('bottom', "Time")
self.plot_rtd.setLabel('left', "Distance (mm)")
self.plot_rtd.enableAutoRange()
self.plot_rtd.invertY(True)
self.curve_rtd = self.plot_rtd.plot(pen='r')

graph_panel.addWidget(self.plot_profile, 2)
graph_panel.addWidget(self.plot_rtd, 1)

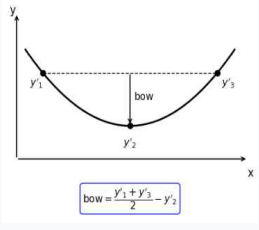
layout.addLayout(graph_panel, 4)

self.update_ports()
```

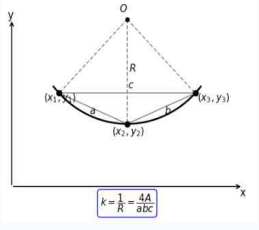
실시간 거리값 출력

03 작품설명

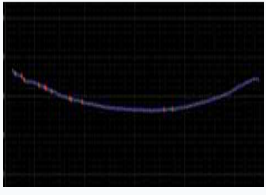
● 평탄도 측정 공식



● 곡률 측정 공식



● 불량 판정 기준



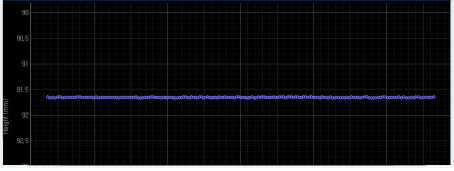
임계값 = 0.05mm 초과

04 프로젝트 결과

CASE 1. 평평함(FLAT)



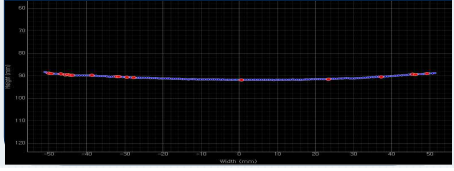
곡률(R) : 5910.6 mm 평탄도 : -0.019 mm



CASE 2. 조금 휨(LOW)



곡률(R) : 661.8 mm 평탄도 : -1.434 mm

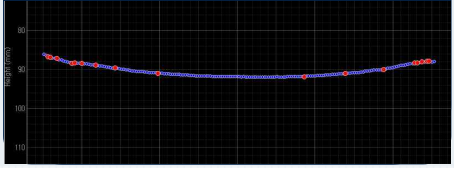


04 프로젝트 결과

CASE 3. 중간 휨(MID)



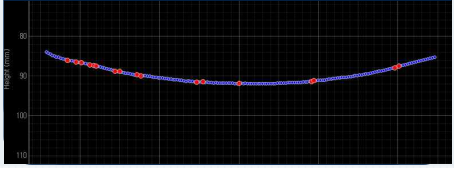
곡률(R) : 337.8 mm 평탄도 : -3.237 mm

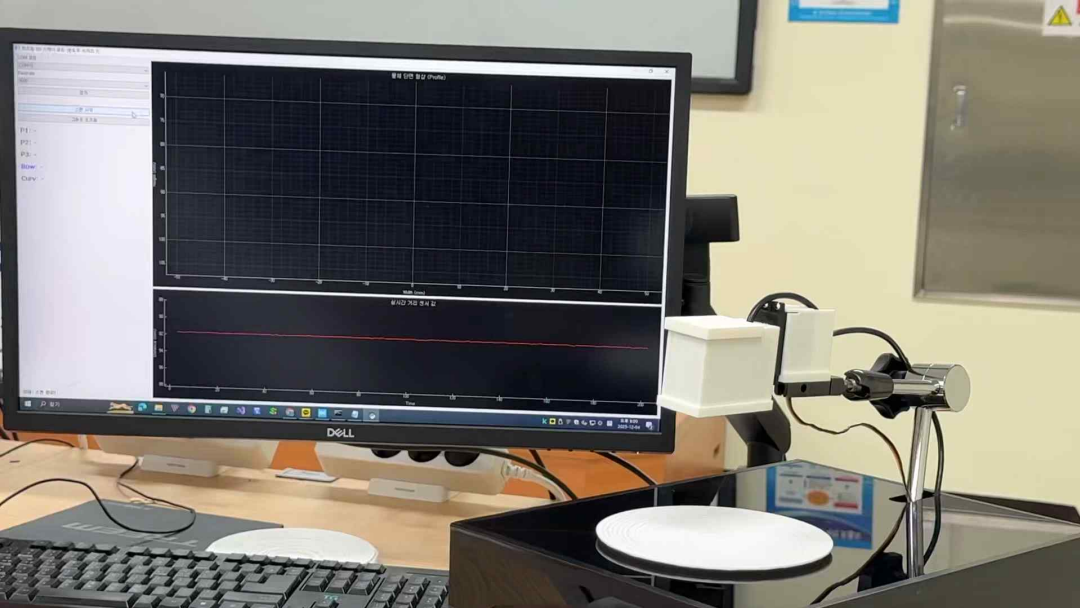


CASE 4. 많이 휨(HIGH)



곡률(R) : 202.6 mm 평탄도 : -5.534 mm




DELL


웨이퍼 단면 형상 (Profile)
Width (mm)
실시간 거리 변화 값
Time
P1: --
P2: --
P3: --
Bow: --
Curv: --
DELL

05 결과분석 및 토의



정확도

설계 목표
■ 측정 오차율 ±1.0% 이내,
반복 정밀도 0.1mm 이내

최종 결과
■ 평균 오차율 ± 0.03%,
반복 정밀도 0.03mm



성능 개선

STM32F411CEU6
■ Voltage Level - 0V~3.3V

BD-100 (레이저 변위 센서)
■ Voltage Level - 1V~5V
STM32 내부 ADC 사용으로 인한 손실 발생
⟹ 정확도 저하
■ 8회 오버 샘플링을 통해 분해능
13.43um⟹ 4.75um으로 정확도 저하 최소화



기대 효과

■ **Flexible Display**와 같은 곡률 측정이 필수적인 제품에 확대하여 사용 가능

■ 측정 데이터를 그래프와 수치로 제공하여 공정개선을 위한 엔지니어의 판단에 도움

06 느낀점 및 애로사항

이론과 실무의 차이

설계했던 계획보다 실제 구현 시 모터의 미세 진동이나 센서 노이즈와 같은 물리적 변수에 부딪힘

H/W와 S/W 통합 과정에서 수많은 시행착오 발생

끈기 있는 최적화

회로 수정과 코드 최적화를 반복하며 문제를 해결

결과적으로 3D 프린팅 시편 표면의 '미세한 적층 결' 까지 잡아내는 수준의 측정 정밀도를 구현

엔지니어 역량 확보

단순 조립을 넘어 하나의 완성된 시스템을 설계하고 검증하는 전체 과정을 통해 엔지니어로써의 역량 확보

호서대학교
반도체특성화대학
Hoseo Semiconductor-Specialized University

감사합니다